

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«Национальный исследовательский университет ИТМО»

ПРАКТИЧЕСКАЯ РАБОТА №1
по дисциплине
«Администрирование компьютерных сетей»

Тема работы:
HA Postgres Cluster

Студенты:

1. Катков Алексей
2. Турищев Артём
3. Камалов Руслан
4. Филиппов Владимир
5. Клименко Богдан
6. Митрофанов Георгий

Преподаватель: Самохин Н. Ю.

Санкт-Петербург
2026

1 Введение

Целью данной лабораторной работы является изучение принципов построения отказоустойчивых кластеров баз данных PostgreSQL с использованием Patroni, Docker и Zookeeper.

В ходе работы был развёрнут кластер PostgreSQL с конфигурацией master–slave, обеспечивающий репликацию данных и автоматическое управление ролями узлов.

2 Постановка задачи

В рамках лабораторной работы необходимо:

- Развернуть кластер PostgreSQL с использованием Docker Compose;
- Настроить Patroni для управления ролями master и slave;
- Использовать Zookeeper для координации кластера;
- Проверить работу репликации данных;
- Продемонстрировать отказоустойчивость системы.

3 Архитектура решения

Кластер состоит из следующих компонентов:

- PostgreSQL master;
- PostgreSQL slave;
- Patroni;
- Zookeeper;
- Docker и Docker Compose.

Все сервисы развёрнуты в отдельных контейнерах и объединены общей сетью Docker.

4 Развёртывание кластера

Для запуска кластера использовалась команда:

```
1 docker-compose up -d
```

В результате были успешно созданы и запущены контейнеры PostgreSQL master, PostgreSQL slave и Zookeeper.

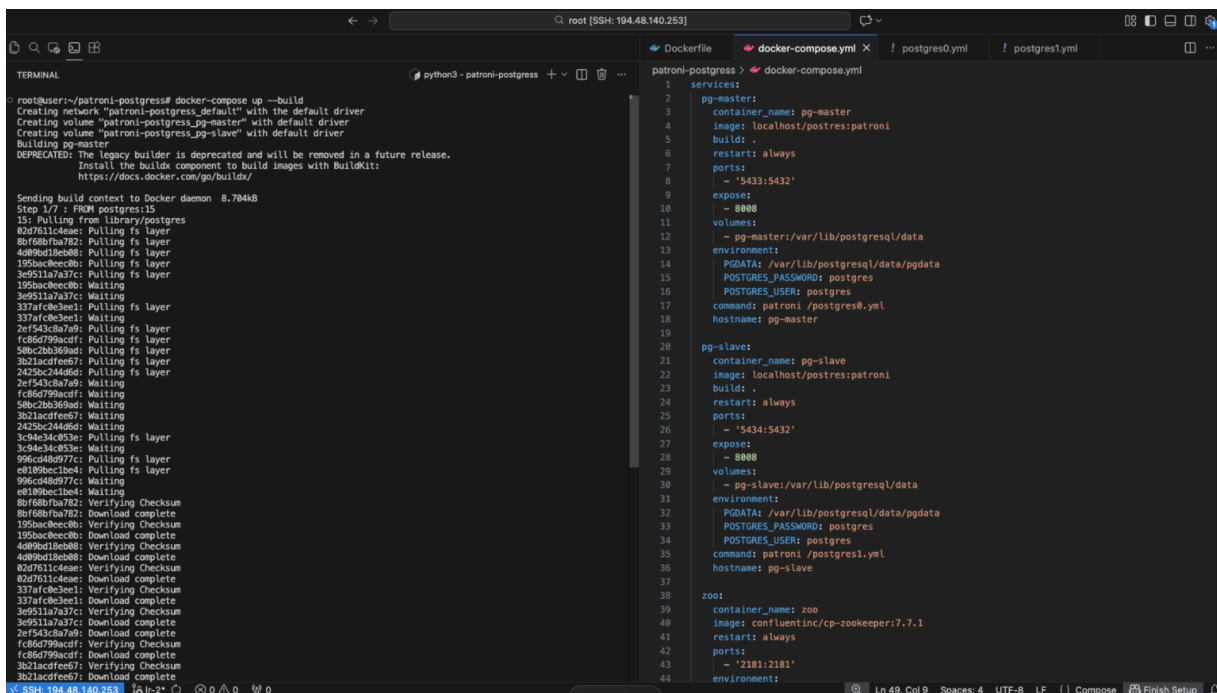


Рис. 1: Сборка и запуск контейнеров Docker

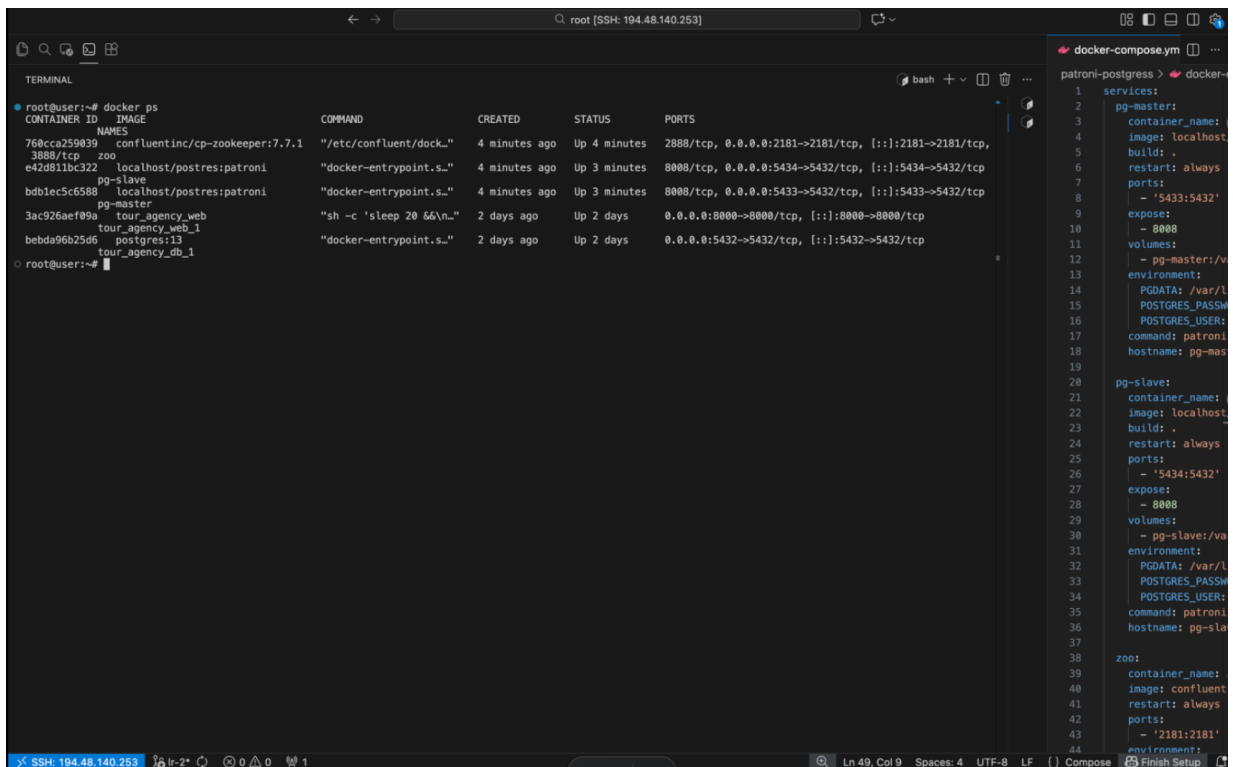


Рис. 2: Список запущенных контейнеров Docker

5 Подключение к базе данных

Для проверки работы кластера использовался клиент DBeaver. Было выполнено подключение как к master-, так и к slave-узлу PostgreSQL.

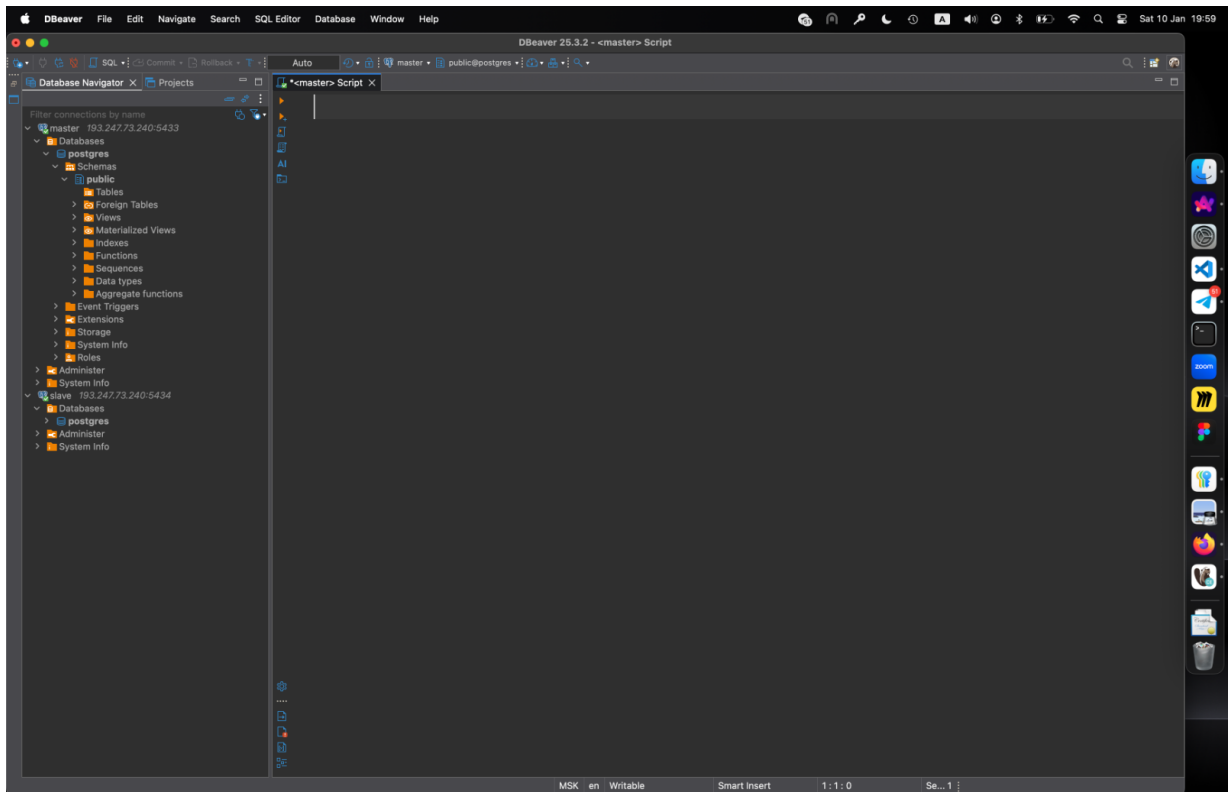


Рис. 3: Подключение к master и slave в DBeaver

6 Проверка репликации

На master-узле была создана тестовая таблица:

```
1 CREATE TABLE my_first_replication (  
2     id INT,  
3     my_data VARCHAR,  
4     my_comment VARCHAR  
5 );
```

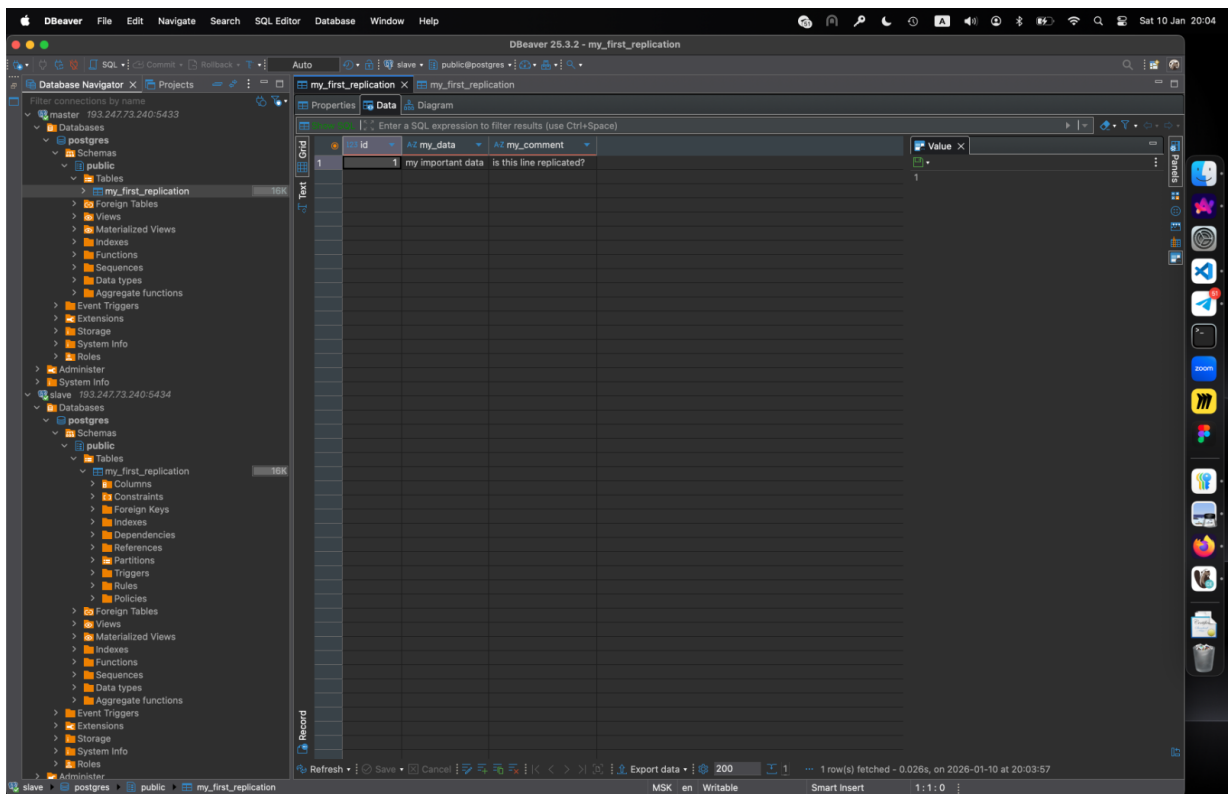


Рис. 4: Создание таблицы на master-узле

После этого в таблицу были добавлены данные:

```
1 INSERT INTO my_first_replication VALUES (  
2     1,  
3     'my important data',  
4     'is this line replicated?'  
5 );
```

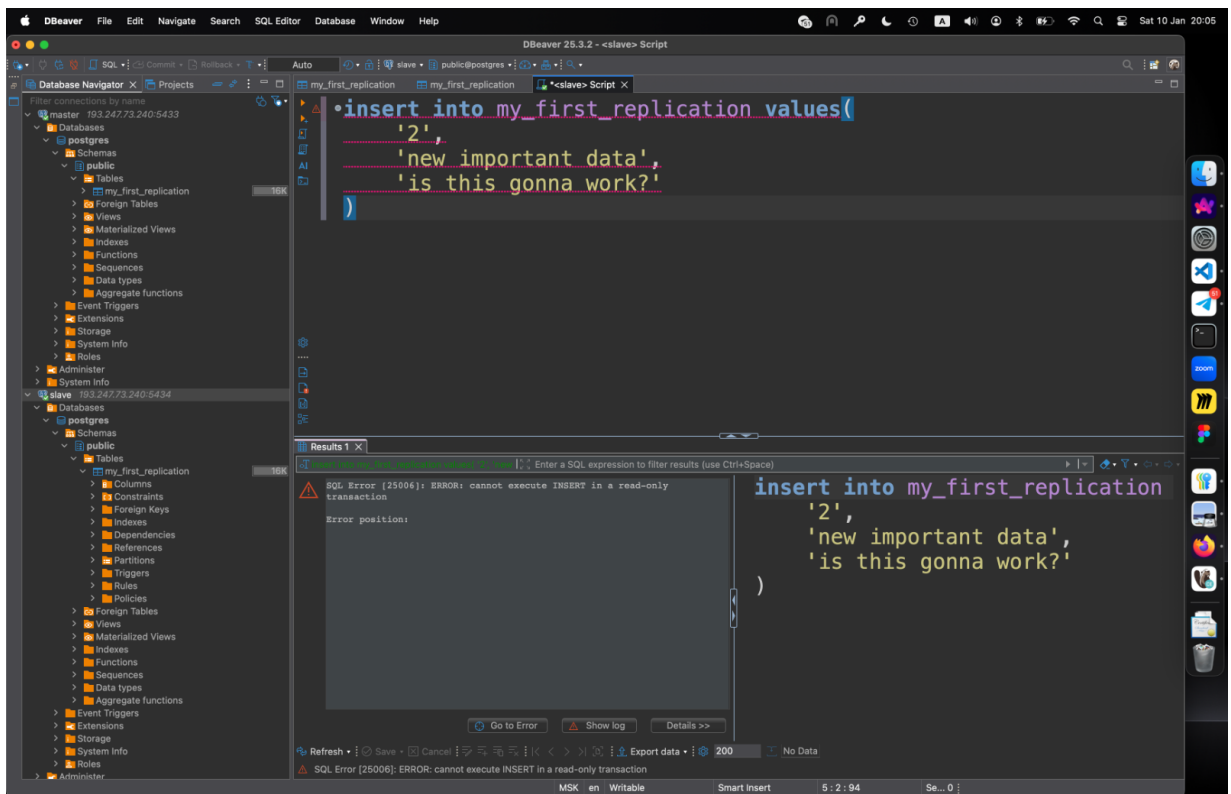


Рис. 5: Добавление данных на master-узле

На slave-узле была подтверждена репликация данных, при этом попытка записи завершилась ошибкой, что подтверждает режим «только чтение».

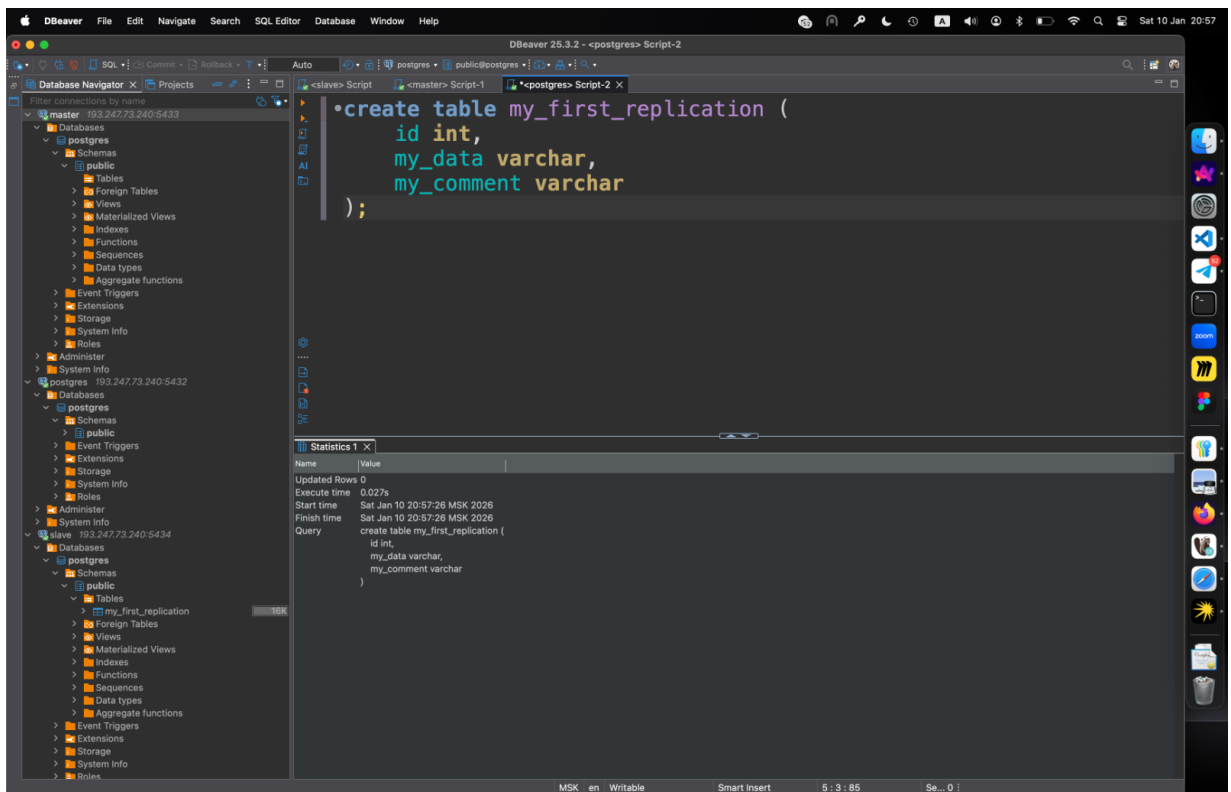


Рис. 6: Ошибка записи на slave-узле (read-only)

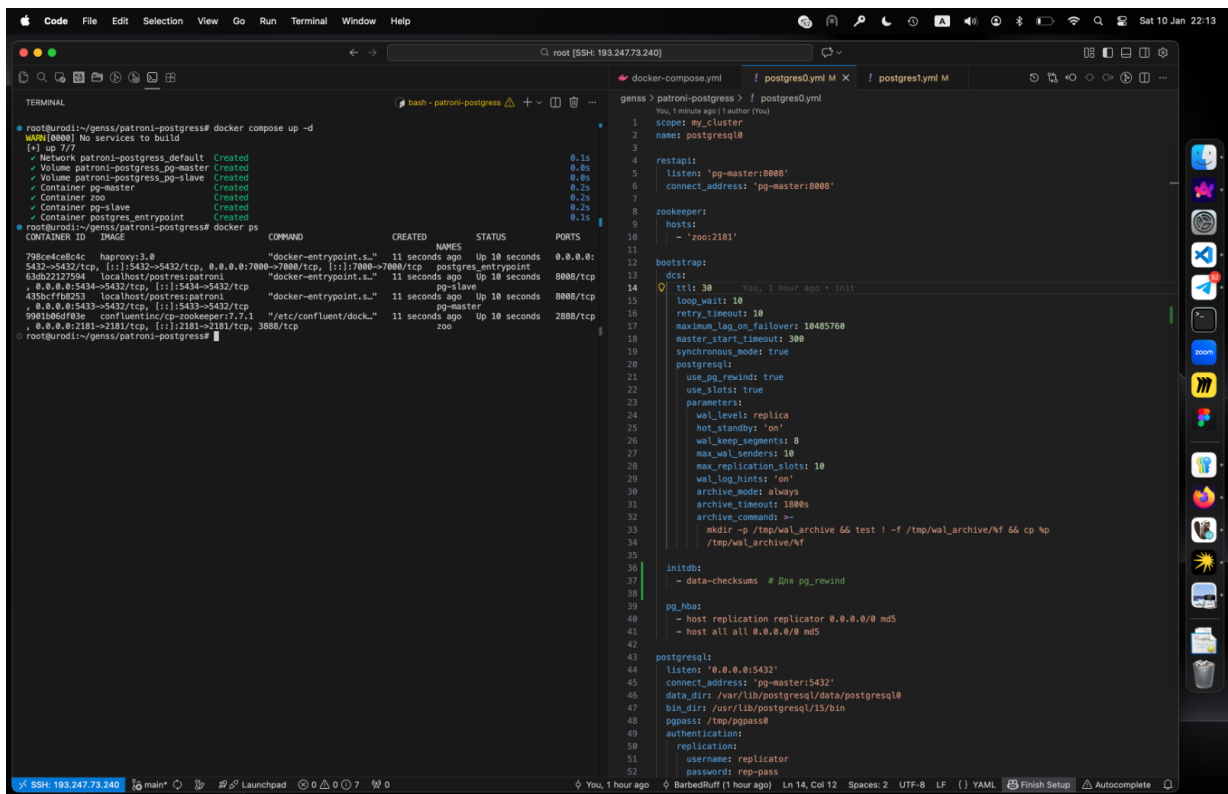


Рис. 7: Наличие реплицированных данных на slave

7 Проверка отказоустойчивости

Для проверки отказоустойчивости был остановлен контейнер master-узла. После этого Patroni автоматически перераспределит роли в кластере.

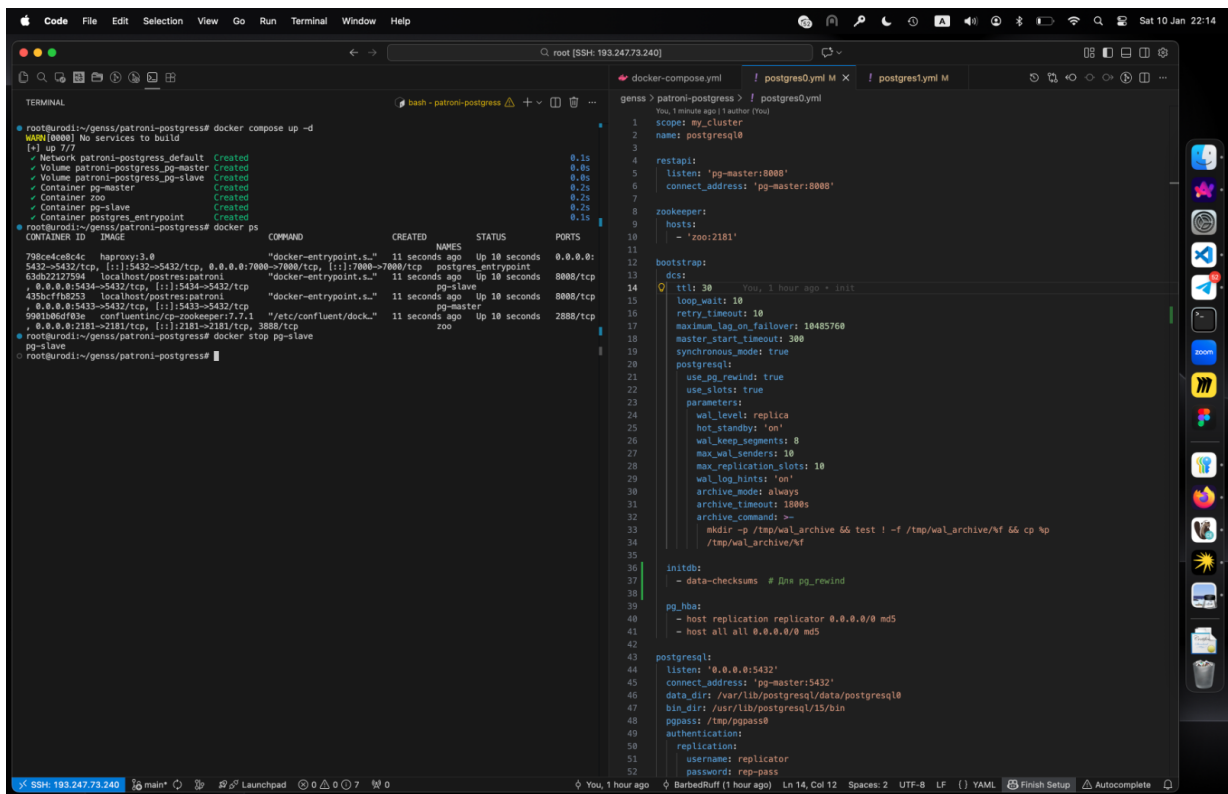


Рис. 8: Остановка master-контейнера

Было подтверждено, что кластер остаётся работоспособным, а доступ к данным сохраняется.

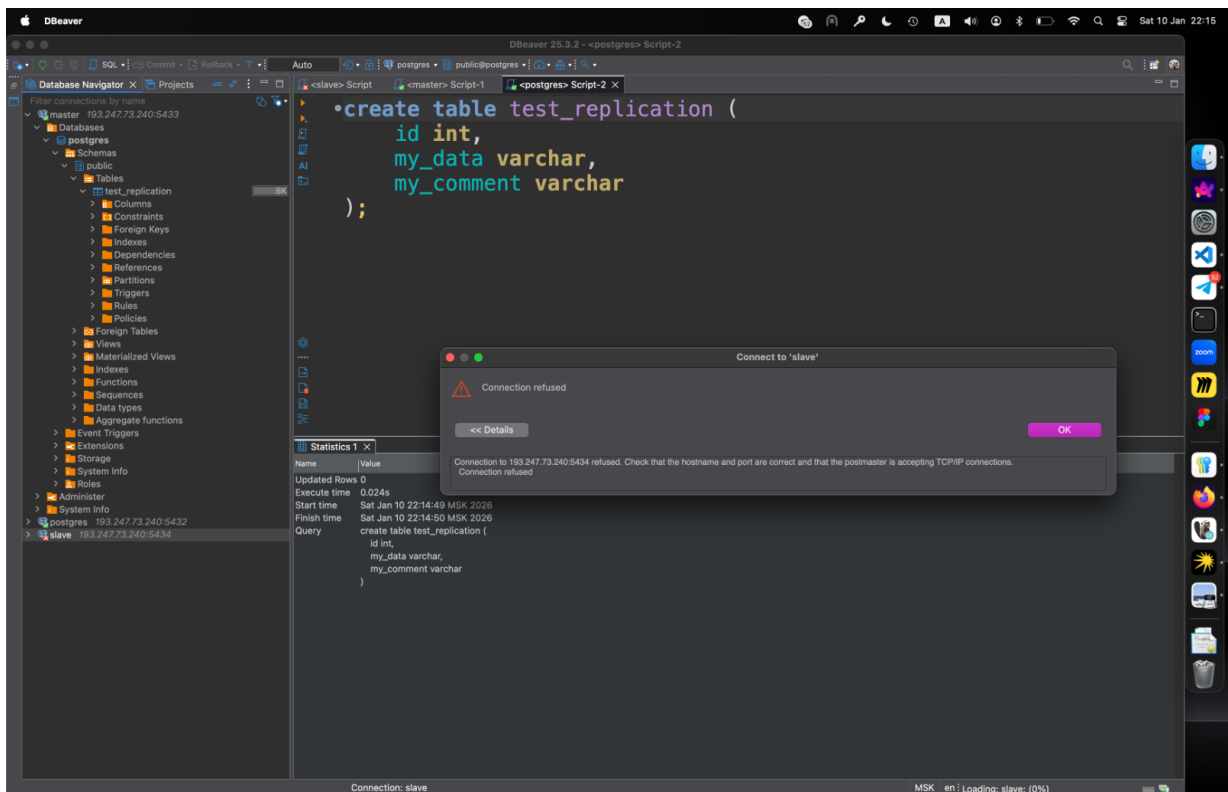


Рис. 9: Работа кластера после отказа master

8 Дополнительная проверка

Была выполнена попытка создания новой таблицы и анализа состояния подключений.

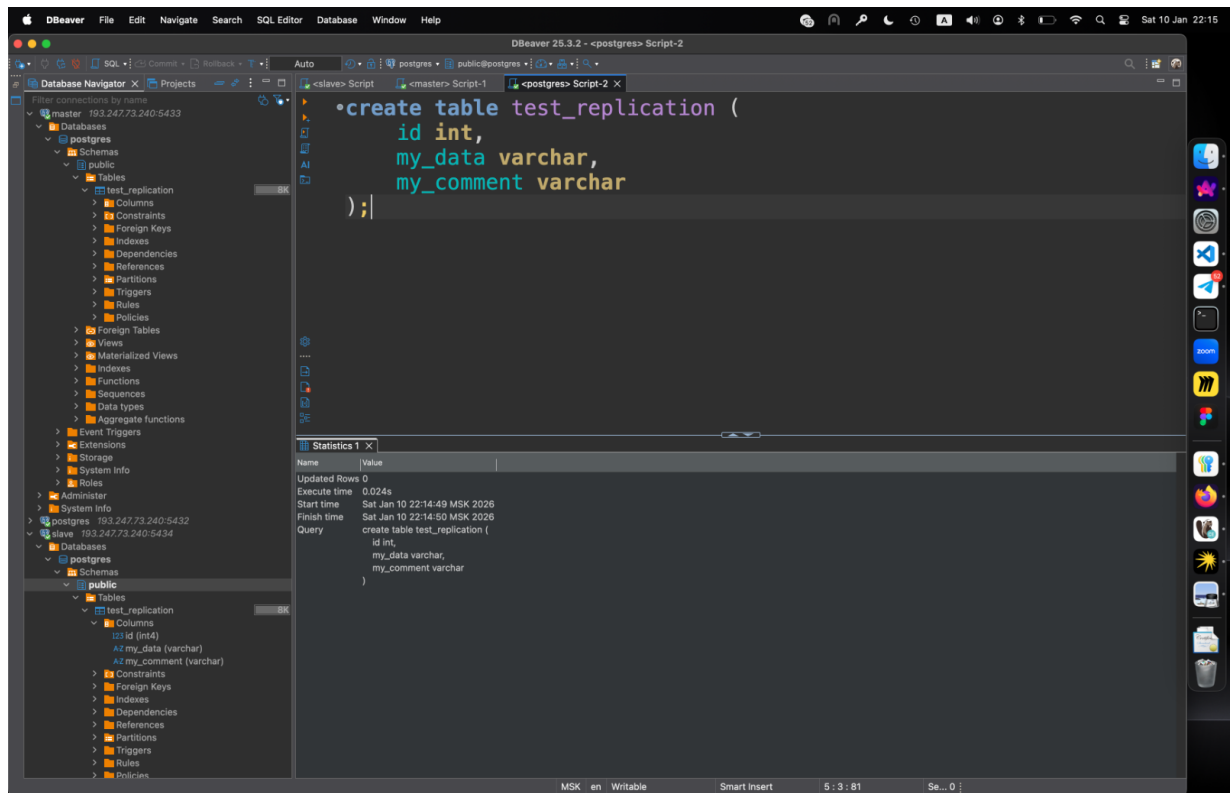


Рис. 10: Дополнительная проверка состояния БД

9 Заключение

В ходе лабораторной работы был успешно развёрнут отказоустойчивый кластер PostgreSQL с использованием Docker, Patroni и Zookeeper.

Была подтверждена корректная работа репликации данных, а также автоматическое восстановление работоспособности при отказе master-узла.